Functions Part 1

Section 1
Chapter 4

An Introduction to Programming Using Python
David I. Schneider

Functions

- Topics
  - Built in functions
  - User defined functions
    - Parameters and arguments
  - Scope of variables

What are Functions?

- Used to decompose complex tasks into smaller sub-problems.
  - Makes code more readable
  - Makes code reusable – eliminates repetitive code
- Every function returns a value
  - either a single value or tuple of values
  - can leave function at any point using *return* statement
  - no *return* statement or no argument for *return* → returns **None**

Built-in Functions

- Like miniature programs
  - Receive input
  - Process the input
  - Have output
- Table 4.1 Some Python built-in functions.

Function	Example	Input	Output
int	int(2.6) is 2	number	number
chr	chr(65) is “A”	number	string
ord	ord('A') is 65	string	number
round	round(2.34, 1) is 2.3	number, number	number

Built-in Functions

- Output of functions is a single value
  - Function is said to return its output
- Items inside parentheses called arguments
- Examples:

```
num = int(3.7)          # literal as an argument

num1 = 2.6
num2 = int(num1)        # variable as an argument

num1 = 1.3
num2 = int(2 * num1)    # expression as an argument
```

Built-in Functions

- Defined in the standard Python library
  - Can be called from any Python program.
- Examples:
  - round(6.291, 1) → 6.3
  - int('23') → 23
  - ord('c') → 99
  - input('Enter your name')→ String entered by user
  - print('Hello world!') → None

User-defined Functions

- Defined by statements of the form

```
def functionName(par1, par2, ...):
    indented block of statements()
    return expression
```

  - defined using the keyword def
  - par1, par2 are variables (called parameters)
  - Header must end with colon
  - Each statement in block indented below header
  - Expression evaluates to a literal of any type

User-defined Functions

- Passing parameters
  - We consider here pass by position
  - Arguments in calling statement matched to the parameters in function header based on order
- **Parameters** and **return** statements optional in function definitions
- Function names should describe the role performed

```
def trianglePerimeter(s1, s2, s3):
    perimeter= s1 + s2 + s3
    return perimeter
```

Function Call

```
def trianglePerimeter(s1, s2, s3):
    perimeter= s1 + s2 + s3
    return perimeter
```

- A function may be called (or invoked) from elsewhere in the program.
  - The variables/values in the function call are referred to as arguments
- Sample function call: sides = [5, 8, 2]
  - result = trianglePerimeter(5, 8, 2)
  - result = trianglePerimeter(sides[0], sides[1], sides[2])
  - result = 15

Functions Having One Parameter

```
def fahrenheitToCelsius(t):
    ## Convert Fahrenheit temperature to Celsius.
    convertedTemperature = (5 / 9) * (t - 32)
    return convertedTemperature

def firstName(fullName):
    ## Extract the first name from a full name.
    firstSpace = fullName.index(" ")
    givenName = fullName[:firstSpace]
    return givenName
```

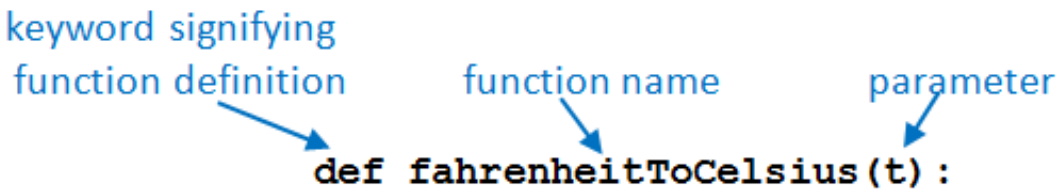


FIGURE 4.1 Header of the fahrenheitToCelsius function

Functions Having One Parameter

- Example 1: Program uses the function *fahrenheitToCelsius*

```
def fahrenheitToCelsius(t):
    ## Convert Fahrenheit temperature to Celsius.
    convertedTemperature = (5 / 9) * (t - 32)
    return convertedTemperature

fahrenheitTemp = eval(input("Enter a temperature in degrees Fahrenheit: "))
celsiusTemp = fahrenheitToCelsius(fahrenheitTemp)
print("Celsius equivalent:", celsiusTemp, "degrees")
```

[Run]

```
Enter a temperature in degrees Fahrenheit: 212
Celsius equivalent: 100.0 degrees
```

Passing a Value to a Function

- If the argument in a function call is a variable
  - Object pointed to by the argument variable (not the argument variable itself) passed to a parameter variable
  - Object is immutable, there is no possibility that value of the argument variable will be changed by a function call

Passing a Value to a Function

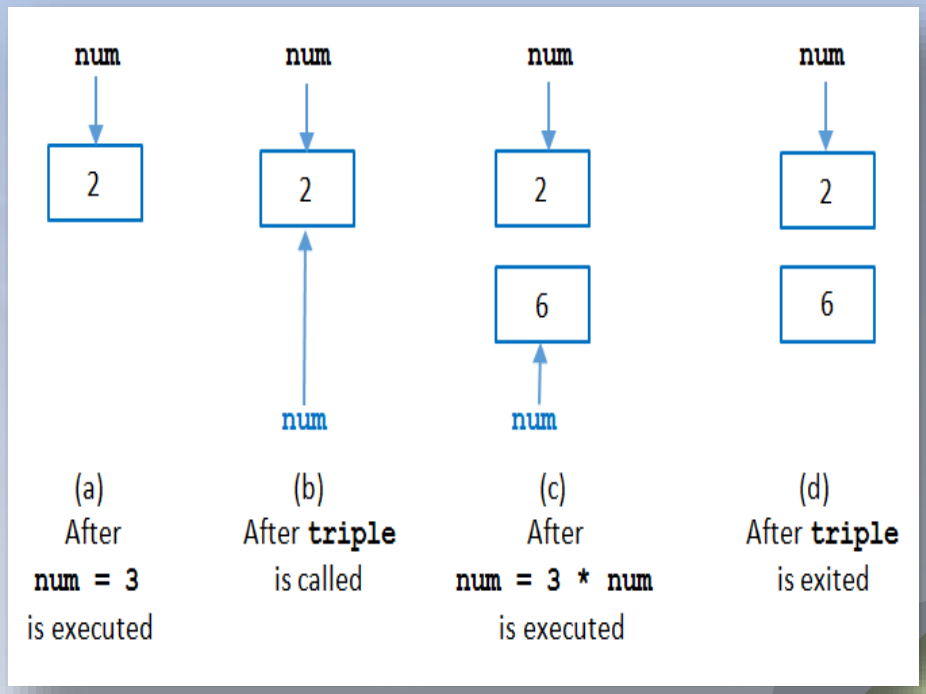
- Example 3: Program shows there is no change in the value of the argument

```
def triple(num):
    num = 3 * num
    return num

num = 2
print(triple(num))
print(num)
```

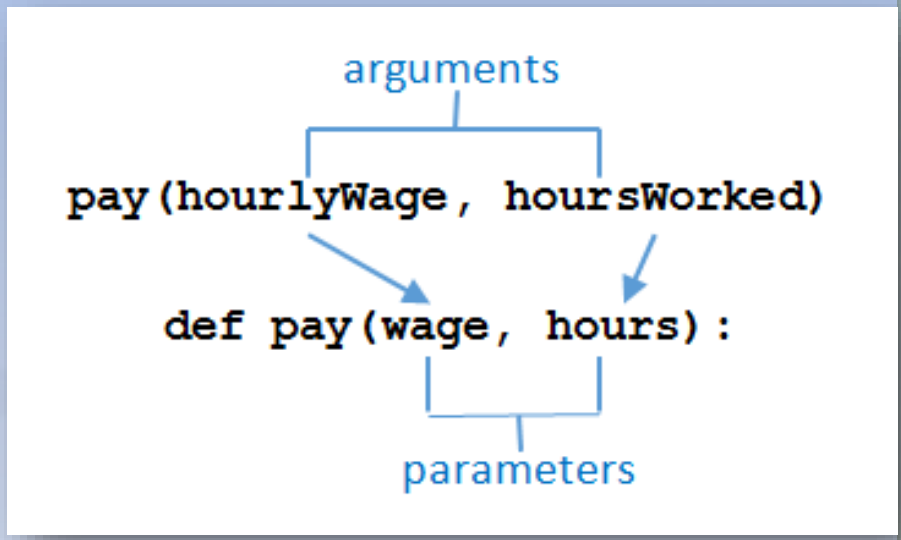
[Run]

```
6
2
```



Functions Having Several Parameters

- Must be the same number of arguments as parameters in the function
- Data types of arguments' values must be compatible with data types expected by the parameters
  - Must also be in the same order



Functions Having Several Parameters

- Example 4: Program uses the function *pay*.

```
def pay(wage, hours):
    ## Calculate weekly pay with time-and-a-half for overtime.
    if hours <= 40:
        amount = wage * hours
    else:
        amount = (wage * 40) + ((1.5) * wage * (hours - 40))
    return amount

## Calculate a person's weekly pay.
hourlyWage = eval(input("Enter the hourly wage: "))
hoursworked = eval(input("Enter the number of hours worked: "))
print("Earnings: ${0:,.2f}".format(pay(hourlyWage, hoursWorked)))
```

[Run]

```
Enter the hourly wage: 24.50
Enter the number of hours worked: 45
Earnings: $1,163.75
```

Boolean- and List-valued Functions

- Example 6: Program uses a Boolean-valued function to determine whether input is a vowel word

```
def isVowelWord(word):
    word = word.upper()
    vowels = ('A', 'E', 'I', 'O', 'U')
    for vowel in vowels:
        if vowel not in word:
            return False
    return True

## Determine if a word contains every vowel.
word = input("Enter a word: ")
if isVowelWord(word):
    print(word, "contains every vowel.")
else:
    print(word, "does not contain every vowel.")
```

[Run]

```
Enter a word: Education
Education contains every vowel.
```

Functions that do not Return Values

- Example 8: Program displays three verses of children's song.

```
def oldMcDonald(animal, sound):
    print("Old McDonald had a farm. Eyi eyi oh.")
    print("And on his farm he had a", animal + ".", "Eyi eyi oh.")
    print("With a", sound, sound, "here, and a", sound, sound, "there.")
    print("Here a", sound + ",", "there a", sound + ",", \
          "everywhere a", sound, sound + ".")
    print("Old McDonald had a farm. Eyi eyi oh.")

## Old McDonald Had a Farm
oldMcDonald("lamb", "baa")
print()
oldMcDonald("duck", "quack")
print()
oldMcDonald("cow", "moo")
```

[Run]

```
Old McDonald had a farm. Eyi eyi oh.
And on his farm he had a lamb. Eyi eyi oh.
With a baa baa here, and a baa baa there.        ... etc. ...
```

Scope of Variables

- Variable created inside a function can only be accessed by statements inside that function
  - Ceases to exist when the function is exited
- Variable is said to be local to function or to have local scope
- If variables created in two different functions have the same name
  - They have no relationship to each other

Scope of Variables

- Example 10: Variable x in the function main, variable x in the function trivial are different variables

```
def main():
    ## Demonstrate the scope of variables.
    x = 2
    print(str(x) + ": function main")
    trivial()
    print(str(x) + ": function main")

def trivial():
    x = 3
    print(str(x) + ": function trivial")

main()
```

[Run]

```
2: function main
3: function trivial
2: function main
```

Scope of Variables

- Example 11: Variable x created in function main not recognized by function trivial.

```
def main():
    ## Demonstrate the scope of local variables.
    x = 5
    trivial()

def trivial():
    print(x)

main()
```

Scope of Variables

- Scope of a variable is the portion of the program that can refer to it
- To make a variable global, place assignment statement that creates it at top of program.
  - Any function can read the value of a global variable
  - Value cannot be altered inside a function unless

```
global globalVariableName
```

Scope of Variables

- Example 12: Program contains a global variable

```
x = 0   # Declare a global variable.

def main():
    ## Demonstrate the scope of a global variable.
    print(str(x) + ": function main")
    trivial()
    print(str(x) + ": function main")

def trivial():
    global x
    x += 7
    print(str(x) + ": function trivial")

main()
```

[Run]

```
0: function main
7: function trivial
7: function main
```

End

Section 1
Chapter 4

An Introduction to Programming Using Python
David I. Schneider